

X C P C T e m

BHUACM

2 0 2 年 1 0 月 1 8 日

目 录

1 动 态 规 划	5
1 . 0 1 背 包	5
1 . S O S D P	5
1 . m 元 进 制 优 升 子 序 列	5
1 . 背 包	6
1 . 全 错 排	6
1 . 区 间 p	6
1 . 四 边 形 不 等 d p	6
1 . 多 重 背 包	7
1 . 完 全 背 包	7
1 . 数 位 d p	7
1 . 最 大 子 段 和	8
1 . 最 长 上 升 子 序 列	8
1 . 最 长 公 共 子 串	8
1 . 最 长 公 共 子 序 列	9
2 图 论	9
2 . 0 1 B F S	9
2 . B e l l m a n - f o r d	9
2 . D i j k s t r a	1
2 . F l o y d	1
2 . F l o 找 最 小	1
2 . K r u s k a l	1
2 . P r	1
2 . S P	1
2 . d f 判 二 分 图	1
2 . 匈 牙 利 算 法	1
2 . 同 余 最 短 路	1
2 . 哈 密 顿 路 径	1

2.13	并查集判断二分图	16
2.14	强连通分量	16
2.15	找割点	17
2.16	拓扑排序	17
2.17	最长路	18
2.18	点双联通分量	18
2.19	缩点	19
2.20	网络流	19
2.21	边双联通分量	24
2.22	链式前向星	25
3	字符串	25
3.1	AC 自动机	25
3.2	hash	29
3.3	kmp	29
3.4	字典树	30
3.5	马拉车	31
4	数学	31
4.1	MillerRabin	31
4.2	Miller_Rabin + Pollard_Rho 分解质因子	32
4.3	Pollard_Rho	32
4.4	中国剩余定理	33
4.5	区间筛	33
4.6	卢卡斯定理	34
4.7	因子个数 $O(n)$	34
4.8	因子个数 $O(n \log n)$	34
4.9	因子和 $O(n)$	35
4.10	因子和 $O(n \log n)$	35
4.11	威尔逊定理	36
4.12	扩展欧几里得	36
4.13	整除分块	36
4.14	欧拉函数	37
4.15	欧拉筛	37
4.16	求 x 的所有因子	38
4.17	求区间内和 k 互质数的个数	38
4.18	矩阵	39
4.19	等差乘等比求和	40
4.20	等差数列求和	40
4.21	等比数列求和	40
4.22	线性基	40
4.23	线性求逆元	42

4.24	组合数	42
4.25	莫比乌斯函数	42
4.26	费马小定理	43
4.27	降幂	43
4.28	预处理 $\log 2$	43
5	数据结构	43
5.1	LCA	43
5.2	ST 表	44
5.3	cdq 分治求三维偏序	44
5.4	dsu on tree	46
5.5	三分	46
5.6	主席树前 k 大之和	46
5.7	主席树区间第 k 小	48
5.8	二维 ST 表	49
5.9	区间异或和最值	50
5.10	单调栈	51
5.11	单调队列找最值	51
5.12	对顶堆	51
5.13	带权并查集	52
5.14	并查集	52
5.15	归并排序	52
5.16	扫描线求面积并	53
5.17	树状数组	54
5.18	树的直径	55
5.19	树的重心	55
5.20	线段树	56
5.21	莫队	57
5.22	预处理 LCA	58
6	杂	59
6.1	$\oplus$ 异或期望	59
6.2	map	59
6.3	oset	59
6.4	$\sqrt{x}$ 向上取整	59
6.5	上取整下取整	60
6.6	二分图	60
6.7	全排列	60
6.8	判断一个数是不是二的幂次	60
6.9	博弈 1	60
6.10	去重	61
6.11	异或	61

6.12 期望 1	61
6.13 矩形相交	61
6.14 规律 1	61
6.15 质数差	61
6.16 随机模数	62

1 动态规划

1.1 01 背包

```

1  for(int i = 1; i <= n; i++)
2  {
3      for (int j = m; j >= w[i]; j--)
4          dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
5  }
```

1.2 SOSDP

```

1  //SOSDP(子集和DP) x二进制长度为t dp[x] 从 x 的所有二进制子集转移过来
2
3  for(int i = 0; i < t; i++)
4  {
5      for(int j = 0; j < (1 << t); j++)
6      {
7          if(j >> i & 1)
8          {
9              dp[j] += dp[j ^ (1 << i)];
10         }
11     }
12 }
```

1.3 m 元单调上升子序列

```

1  // m元单调上升子序列
2  cin >> n >> m;
3  for (int i = 1; i <= n; i++)
4  {
5      cin >> arr[i];
6      brr[i] = arr[i];
7  }
8  sort(brr+1, brr+1+n);
9  n1 = unique(brr+1, brr+1+n) - (brr+1);
10 for (int i = 1; i <= n; i++)
11 {
12     arr[i] = lower_bound(brr+1, brr+1+n1, arr[i]) - brr;
13     dp[1][i] = 1;
14 }
15 for (int i = 2; i <= m; i++)
16 {
17     memset(t, 0, sizeof(t));
18     for (int j = 1; j <= n; j++)
19     {
20         dp[i][j] = ask(arr[j]-1);
21         add(arr[j], dp[i-1][j]);
22     }
23 }
24 ll ans = 0;
```

```

25 for (int i = 1; i <= n; i++)
26     ans = (ans + dp[m][i]) % mod;

```

1.4 二进制优化多重背包

```

1  for (int i = 1; i <= n; i++)
2  {
3      int w, v, p;
4      cin >> w >> v >> p;
5      for (int j = 1; j <= p; j = j << 1)
6      {
7          arr.push_back({j*w, j*v});
8          p -= j;
9      }
10     if(p)
11         arr.push_back({p*w, p*v});
12 }
13 for (int i = 0; i < arr.size(); i++)
14 {
15     for (int j = m; j >= arr[i].first; j--)
16         dp[j] = max(dp[j], dp[j - arr[i].first] + arr[i].second);
17 }

```

1.5 全错排

```

1  dp[1] = 0;
2  dp[2] = 1;
3  for (int i = 3; i <= 20; i++)
4  {
5      dp[i] = (i-1) * (dp[i-1] + dp[i-2]);
6  }

```

1.6 区间 dp

```

1  for (int len = 2; len <= n; len++)
2  {
3      for (int l = 1; l + len - 1 <= n; l++)
4      {
5          int r = l + len - 1;
6          for (int k = l; k < r; k++)
7              ;
8      }
9  }

```

1.7 四边形不等式优化 dp

```

1  for (int k = 1; k < r; k++)
2      dp[l][r] = min(dp[l][k] + dp[k+1][r]) + w(l, r);
3  // 优化为:

```

```

4 for (int k = m[l][r-1]; k <= m[l+1][r]; k++)
5     dp[l][r] = min(dp[l][k] + dp[k+1][r]) + w(l,r);

```

1.8 多重背包

```

1 for (int i = 1; i <= n; i++)
2 {
3     for (int j = 1; j <= m; j++)
4     {
5         for (int k = 0; k < s[i]; k++)
6         {
7             if (j < k*w[i])
8                 dp[i][j] = dp[i-1][j];
9             else
10                dp[i][j] = max(dp[i-1][j], dp[i-1][j - k*w[i]] + k*v[i]);
11        }
12    }
13 }

```

1.9 完全背包

```

1 for (int i = 1; i <= n; i++)
2 {
3     for (int j = w[i]; j <= m; j++)
4         dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
5 }

```

1.10 数位 dp

```

1 //limit 当前位限制 lead 前导0
2 ll ans[20];
3 ll arr[50];
4 ll dp[50][2][2][50];
5 int digit;
6 ll dfs(int pos, int limit, int lead, int sum)
7 {
8     ll ans = 0;
9     if (!pos)
10        return sum;
11     if (dp[pos][limit][lead][sum] != -1 && !limit && !lead)
12        return dp[pos][limit][lead][sum];
13     for (int i = 0; i <= (limit ? arr[pos] : 9); i++)
14     {
15         if (lead && i == 0)
16             ans = ans + dfs(pos-1, limit && i == arr[pos], 1, sum);
17         else
18             ans = ans + dfs(pos-1, limit && i == arr[pos], 0, sum + (i == digit));
19     }
20     if (!lead && !limit)
21        dp[pos][limit][lead][sum] = ans;

```

```

22     return ans;
23 }
24 ll f(ll x)
25 {
26     memset(dp,-1,sizeof(dp));
27     int cnt = 0;
28     while(x)
29     {
30         arr[++cnt] = x % 10;
31         x /= 10;
32     }
33     return dfs(cnt,1,1,0);
34 }
35 void solve()
36 {
37     ll a,b;
38     cin >> a >> b;
39     cout << f(b) - f(a-1) << ' ';
40 }

```

1.11 最大子段和

```

1 //mx[i] 代表考虑前i个数的最大子段和
2 for (int i = 1;i <= n;i++)
3 {
4     dp[i] = max(dp[i-1] + arr[i],arr[i]);
5     mx[i] = max(mx[i-1],dp[i]);
6 }

```

1.12 最长上升子序列

```

1 int len = 0; //len是答案
2 for (int i = 1;i <= n;i++)
3 {
4     if(dp[len] < arr[i])
5         dp[++len] = arr[i];
6     else
7         *lower_bound(dp+1,dp+len+1,arr[i]) = arr[i];
8 }
9 //要求完整的子序列可以权值线段树维护dp

```

1.13 最长公共子串

```

1 for (int i = 1; i <= n1; ++i)
2     for (int j = 1; j <= n2; ++j)
3         if (s1[i - 1] == s2[j - 1])
4             dp[i][j] = dp[i - 1][j - 1] + 1;
5         else
6             dp[i][j] = 0;
7

```



```
8 for (int i = 1; i <= n1; ++i)
9     for (int j = 1; j <= n2; ++j)
10         ma = max(ma, dp[i][j]);
```

1.14 最长公共子序列

```
1 for(int i = 1; i <= n; i++)
2 {
3     for(int j = 1; j <= m; j++)
4     {
5         if(s1[i] == s2[j])
6             dp[i][j] = max(dp[i][j], dp[i-1][j-1] + 1);
7         else
8             dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
9     }
10 }
```

2 图论

2.1 01BFS

```
1 deque<int> q;
2 q.push_front(s);
3 dis[s] = 0;
4 while(!q.empty())
5 {
6     int x = q.front();
7     q.pop_front();
8     if(vis[x]) continue;
9     vis[x] = 1;
10    for(auto it : g[x])
11    {
12        int to = it.to;
13        int w = it.w;
14        if(dis[to] > dis[x] + w)
15        {
16            dis[to] = dis[x] + w;
17            if(w)
18                q.push_back(to);
19            else
20                q.push_front(to);
21        }
22    }
23 }
```

2.2 Bellman-ford

```

1 ll dis[MAXN],last[MAXN],re[MAXN] //dis距离, last用来存上一次的結果。
2 for (int i = 1;i <= n;i++)
3 {
4     for (int j = 1;j <= n;j++)
5         re[i][j] = INF;
6 }
7 void Bellman_ford()
8 {
9     for (int i = 1;i <= n;i++)
10         dis[i] = INF;
11     dis[1] = 0;
12     for (int i = 1;i < n;i++)
13     {
14         memcpy(last,dis,sizeof dis);
15         for (int j = 1;j <= m;j++)
16         {
17             ll fr = edges[j].fr,to = edges[j].to,w = edges[j].w;
18             dis[to] = min(dis[to],last[fr] + w);
19         }
20
21         // for (int j = 1;j <= n;j++)
22         //     re[i][j] = dis[j];
23         // i 是边数
24         // re数组维护了从 1 到 j 使用了 i 条边的最短路
25     }
26
27     // for (int j = 1;j <= cnt;j++)
28     // {
29     //     ll fr = edges[j].fr,to = edges[j].to,w = edges[j].w;
30     //     if(last[fr] + w < dis[to])
31     //     {
32     //         // 有负权回路
33     //     }
34 }

```

2.3 Dijkstra

```

1 void dij(int s)
2 {
3     for (int i = 1;i <= n;i++)
4         dis[i] = INF;
5     dis[s] = 0;
6     priority_queue<PII,vector<PII>,greater<PII>>q;
7     q.push({0,s});
8     while(!q.empty())
9     {
10         int x = q.top().second;
11         q.pop();
12         if(vis[x]) continue;
13         vis[x] = 1;
14         for (auto it : g[x])
15         {
16             int to = it.to;
17             int w = it.w;

```

```

18         if(dis[to] > dis[x] + w)
19         {
20             dis[to] = dis[x] + w;
21             q.push({dis[to],to});
22         }
23     }
24 }
25 }

```

2.4 Floyd

```

1 void floyd()
2 {
3     for (int k = 1;k <= n;k++)
4     {
5         for (int i = 1;i <= n;i++)
6         {
7             for (int j = 1;j <= n;j++)
8                 dis[i][j] = min(dis[i][k]+dis[k][j],dis[i][j])
9         }
10    }
11 }

```

2.5 Floyd 找最小环

```

1 void floyd()
2 {
3     for (int k = 1;k <= n;k++)
4     {
5         for (int i = 1;i < k;i++)
6         {
7             for (int j = 1;j < k;j++)
8             {
9                 if(i!=j)
10                    ans = min(ans,dis[i][j] + mp[i][k] + mp[k][j]);
11            }
12        }
13        for (int i = 1;i <= n;i++)
14        {
15            for(int j = 1;j <= n;j++)
16                dis[i][j] = min(dis[i][k]+dis[k][j],dis[i][j]);
17        }
18    }
19 }

```

2.6 Kruskal

```

1 struct edge
2 {
3     int fr,to,w;

```

```

4 }edges[n];
5 int find(int x)
6 {
7     return x == f[x] ? x : f[x] = find(f[x]);
8 }
9 bool cmp(edge a,edge b)
10 {
11     return a.w < b.w;
12 }
13 void kruskal()
14 {
15     for (int i = 1;i <= n;i++)
16         f[i] = i;
17     sort(edges+1,edges+1+cnt,cmp);
18     for (int i = 1;i <= cnt;i++)
19     {
20         int a = find(edges[i].fr);
21         int b = find(edges[i].to);
22         if(a != b)
23         {
24             ans += edges[i].w;
25             f[a] = b;
26             tmp++;
27             if(tmp == n-1)
28                 return;
29         }
30     }
31     return << orz //不连通
32 }

```

2.7 Prim

```

1 void prim()
2 {
3     for(int i = 1;i <= n;i++)
4         dis[i] = INF;
5     dis[1] = 0;
6     int cnt = 0;
7     priority_queue<PII,vector<PII>,greater<PII>> q;
8     q.push({0,1});
9     while(!q.empty())
10    {
11        int x = q.top().se;
12        ll d = q.top().fi;
13        q.pop();
14        if(vis[x]) continue;
15        vis[x] = 1;
16        ans += d;
17        cnt++;
18        for(auto it : g[x])
19        {
20            int to = it.fi;
21            if(dis[to] > it.se)
22            {

```

```

23         dis[to] = it.se;
24         q.push({dis[to],to});
25     }
26 }
27 }
28 if(cnt == n)
29     cout << ans;
30 else
31     cout << "orz";
32 }

```

2.8 SPFA

```

1 void spfa(int s)
2 {
3     for (int i = 1; i <= n; i++)
4         dis[i] = INF;
5     dis[s] = 0;
6     queue<int> q;
7     q.push(s);
8     vis[s] = 1;
9     while(!q.empty())
10    {
11        int tmp = q.front();
12        q.pop();
13        vis[tmp] = 0;
14        for (int i = head[tmp]; i; i = edges[i].next)
15        {
16            int to = edges[i].to;
17            if(dis[to] > dis[tmp] + edges[i].w)
18            {
19                dis[to] = dis[tmp] + edges[i].w;
20                if(!vis[to])
21                {
22                    q.push(to);
23                    // num[to]++;
24                    // if(num[to] >= n)
25                    //     return false //有负权回路
26                    vis[to] = 1;
27                }
28            }
29        }
30    }
31 }

```

2.9 dfs 判二分图

```

1 bool dfs(int x, int c)
2 {
3     color[x] = c;
4     for (int i = head[x]; i; i = next[i])
5     {

```

```

6         int to = edges[i].to;
7         if(color[to])
8         {
9             if(color[to] == c)
10                return false;
11        }
12        else if(!dfs(to,3 - c))
13            return false;
14    }
15    return true;
16}
17bool f = true;
18for (int i = 1;i <= n;i++)
19{
20    if(!color[i])
21    {
22        if(!dfs(i,1))
23        {
24            f = false;
25            break;
26        }
27    }
28}
29if(f)
30    cout << "YES";
31else
32    cout << "NO";

```

2.10 匈牙利算法

```

1 // 一个二分图中的最大匹配数等于这个图中的最小点覆盖数。
2 int M , N; // M,N分别表示左右两侧集合的元素数量
3 int Map[n][n]; // 链接矩阵存图
4 int p[n]; // 记录当前右侧元素所对应的左侧元素
5 bool vis[n] // 记录右侧元素是否已被访问过
6
7 bool match(int i)
8 {
9     for (int j = 1;j <= N;j++)
10    {
11        if(Map[i][j] && !vis[j])
12        {
13            vis[j] = true;
14            if(p[j] == 0 || match(p[j]))
15            {
16                p[j] = i;
17                return true;
18            }
19        }
20    }
21    return false;
22}
23 int Hungarian()
24 {

```

```

25     int cnt = 0;
26     for (int i = 1; i <= M; i++)
27     {
28         memset(vis, 0, sizeof(vis));
29         if (match(i))
30             cnt++;
31     }
32     return cnt;
33 }

```

2.11 同余最短路

```

1  for(int i = 1; i <= n; i++)
2      cin >> a[i];
3  sort(a+1, a+1+n);
4  for(int i = 0; i < a[1]; i++)
5  {
6      for(int j = 2; j <= n; j++)
7          g[i].pb((i + a[j]) % a[1], a[j]);
8  }
9  dij();
10 ll ans = 0;
11 for(int i = 0; i < a[1]; i++)
12 {
13     // 统计ans
14 }
15 cout << ans;

```

2.12 哈密顿路径

```

1  //A Hamiltonian path is a path which visits every vertex exactly once.
2  //旅行商问题/哈密顿回路/哈密顿路径
3  // 从0号点出发
4  memset(dp, 0x3f, sizeof(dp));
5  dp[1][0] = 0;
6  for(int i = 0; i < (1 << n); i++)
7  {
8      for(int j = 0; j < n; j++)
9      {
10         if(i >> j & 1)
11         {
12             for(int k = 0; k < n; k++)
13             {
14                 if(k != j && (i >> k & 1))
15                     dp[i][j] = min(dp[i][j], dp[i ^ (1 << j)][k] + dis[k][j]);
16             }
17         }
18     }
19 }
20
21 int mi = inf;
22 for(int i = 1; i < n; i++)

```

```

23     mi = min(mi, dp[(1ll << n) - 1][i]) // 哈密顿路径;
24
25     mi = inf;
26     for(int i = 1; i < n; i++)
27         mi = min(mi, dp[(1ll << n) - 1][i] + dis[i][0]) // 哈密顿回路

```

2.13 并查集判断二分图

```

1  for (int i = 1; i <= 2 * n; i++)
2      fa[i] = i;
3  int f = 0;
4  for (int i = 1; i <= n; i++)
5  {
6      int a, b;
7      cin >> a >> b;
8      merge(a, b + n);
9      merge(b, a + n);
10 }
11 for (int i = 1; i <= n; i++)
12 {
13     if(find(i) == find(i + n))
14         return false;
15 }

```

2.14 强连通分量

```

1  stack<int> s;
2  // ins : 是否在栈中 scc: 每个点所属的强连通分量编号 sum: 强连通分量的数量
3  int dfn[n], low[n], ins[n], scc[n], cnt, sum;
4  void tarjan(int x)
5  {
6      low[x] = dfn[x] = ++cnt;
7      ins[x] = 1;
8      s.push(x);
9      for(auto to : g[x])
10     {
11         if(!dfn[to])
12         {
13             tarjan(to);
14             low[x] = min(low[x], low[to]);
15         }
16         else if(ins[to])
17             low[x] = min(low[x], dfn[to]);
18     }
19     if(low[x] == dfn[x])
20     {
21         ++sum;
22         while(!s.empty())
23         {
24             int tmp = s.top();
25             s.pop();
26             ins[tmp] = 0;

```



```

27         scc[tmp] = sum;
28         if(tmp == x)
29             break;
30     }
31 }
32 }
33
34 for (int i = 1; i <= n; i++)
35 {
36     if(!dfn[i])
37         tarjan(i);
38 }

```

2.15 找割点

```

1 void tarjan(int x, int father)
2 {
3     low[x] = dfn[x] = ++cnt;
4     int child = 0;
5     for(auto to : g[x])
6     {
7         if(!dfn[to])
8         {
9             tarjan(to, x);
10            low[x] = min(low[x], low[to]);
11            if(low[to] >= dfn[x])
12                child++;
13        }
14        else if(to != father)
15            low[x] = min(low[x], dfn[to]);
16    }
17
18    if((x == father && child >= 2) || (x != father && child >= 1))
19    {
20        sum++;
21        ans[x] = 1;
22    }
23 }
24 for (int i = 1; i <= n; i++)
25 {
26     if(!dfn[i])
27         tarjan(i, i);
28 }

```

2.16 拓扑排序

```

1 int in[n], arr[n] // in:入度 arr是排序后的数组
2 bool topu(int n)
3 {
4     int cnt = 0;
5     queue<int> q;
6     for (int i = 1; i <= n; i++)

```

```

7      {
8          if(in[i] == 0)
9              q.push(i);
10     }
11     while(!q.empty())
12     {
13         int tmp = q.front();
14         q.pop();
15         arr[++cnt] = tmp;
16         for(int i = head[tmp]; i; i = edges[i].next)
17         {
18             int to = edges[i].to;
19             in[to]--;
20             if(!in[to])
21                 q.push(to);
22         }
23     }
24     return cnt == n;
25 }

```

2.17 最长路

```

1 void spfa(int s)
2 {
3     for (int i = 1; i <= MAXN; i++)
4         dis[i] = -INF;
5     dis[s] = 0;
6     queue<int> q;
7     q.push(s);
8     vis[s] = 1;
9     while(!q.empty())
10    {
11        int tmp = q.front();
12        q.pop();
13        vis[tmp] = 0;
14        for (int i = head[tmp]; i; i = edges[i].next)
15        {
16            int to = edges[i].to;
17            if(dis[to] < dis[tmp] + edges[i].w)
18            {
19                dis[to] = dis[tmp] + edges[i].w;
20                if(!vis[to])
21                {
22                    q.push(to);
23                    vis[to] = 1;
24                }
25            }
26        }
27    }
28 }

```

2.18 点双联通分量

```

1 void tarjan(int x,int father)
2 {
3     low[x] = dfn[x] = ++cnt;
4     s.push(x);
5     for(auto to : g[x])
6     {
7         if(!dfn[to])
8         {
9             tarjan(to,x);
10            low[x] = min(low[x],low[to]);
11        }
12        else if(to != father)
13            low[x] = min(low[x],dfn[to]);
14    }
15    if(low[x] == dfn[x])
16    {
17        sum++;
18        while(!s.empty())
19        {
20            int tmp = s.top();
21            s.pop();
22            scc[tmp] = sum;
23            css[sum] = tmp;
24            if(tmp == x)
25                break;
26        }
27    }
28 }

```

2.19 缩点

```

1 for (int i = 1;i <= n;i++)
2 {
3     for (auto to : g[i])
4     {
5         if(scc[i] != scc[to])
6             add(scc[i],scc[to]);
7     }
8 }

```

2.20 网络流

```

1 // 网络流
2 // 最大流
3 // 最后的残留网络,两点间的反向路径的值就是两点间的实际流量
4 // EK 算法  $O(nm^2)$  算法时间复杂度高,只适用于小图
5 ll flow[n],mp[n][n],pre[n];
6 ll bfs(int s,int t)
7 {
8     for(int i = 1;i <= n;i++) pre[i] = -1;
9     flow[s] = INF;
10    pre[s] = 0;

```

```

11     queue<int> q;
12     q.push(s);
13     while(!q.empty())
14     {
15         int x = q.front();
16         q.pop();
17         if(x == t) break;
18         for(int i = 1; i <= n; i++)
19         {
20             if(i != s && mp[x][i] > 0 && pre[i] == -1)
21             {
22                 pre[i] = x;
23                 q.push(i);
24                 flow[i] = min(flow[x], mp[x][i]);
25             }
26         }
27     }
28     if(pre[t] == -1) return -1;
29     return flow[t];
30 }
31 ll maxflow(int s, int t)
32 {
33     ll ans = 0;
34     while(1)
35     {
36         ll now = bfs(s, t);
37         if(now == -1) break;
38         int cur = t;
39         while(cur != s)
40         {
41             int fa = pre[cur];
42             mp[fa][cur] -= now;
43             mp[cur][fa] += now;
44             cur = fa;
45         }
46         ans += now;
47     }
48     return ans;
49 }
50 void solve()
51 {
52     for(int i = 1; i <= m; i++)
53     {
54         ll a, b, c;
55         cin >> a >> b >> c;
56         mp[a][b] += c;
57     }
58     cout << maxflow(s, t);
59 }
60
61 //Dinic O(n*n*m)
62 struct edge
63 {
64     int to, w, next;
65 }edges[m << 1];
66 int n, m, s, t;

```

```

67 int head[n],cnt = 1;
68 int now[n],dep[n];
69 void add(int from,int to,int w)
70 {
71     edges[++cnt].w = w;
72     edges[cnt].to = to;
73     edges[cnt].next = head[from];
74     head[from] = cnt;
75 }
76 ll bfs()
77 {
78     for(int i = 1;i <= n;i++) dep[i] = inf;
79     dep[s] = 0;
80     now[s] = head[s];
81     queue<int> q;
82     q.push(s);
83     while(!q.empty())
84     {
85         int x = q.front();
86         q.pop();
87         for(int i = head[x];i;i = edges[i].next)
88         {
89             int to = edges[i].to;
90             if(edges[i].w > 0 && dep[to] == inf)
91             {
92                 q.push(to);
93                 now[to] = head[to];
94                 dep[to] = dep[x] + 1;
95                 if(to == t)
96                     return 1;
97             }
98         }
99     }
100     return 0;
101 }
102 ll dfs(int u,ll sum)
103 {
104     if(u == t) return sum;
105     ll now = 0;
106     for(int i = now[u];i&&sum;i = edges[i].next)
107     {
108         now[u] = i;
109         int to = edges[i].to;
110         if(edges[i].w > 0 && dep[to] == dep[u] + 1)
111         {
112             ll k = dfs(to,min(sum,edges[i].w));
113             if(!k) dep[to] = 0;
114             edges[i].w -= k;
115             edges[i^1].w += k;
116             now += k;
117             sum -= k;
118         }
119     }
120     return now;
121 }
122 void solve()

```

```
123 {
124     cin >> n >> m >> s >> t;
125     for(int i = 1; i <= m; i++)
126     {
127         int a, b;
128         ll c;
129         add(a, b, c);
130         add(b, a, 0);
131     }
132     ll ans = 0;
133     while(bfs())
134         ans += dfs(s, inf);
135     cout << ans;
136 }
137
138 //ISAP O(n*n*m)
139 struct edge
140 {
141     int to, next, from;
142     ll w;
143 } edges[m << 1];
144 int n, m, s, t;
145 int head[n], cnt = 1;
146 int now[n], dep[n], pre[n], gap[n];
147 void add(int from, int to, int w)
148 {
149     edges[++cnt].w = w;
150     edges[cnt].from = from;
151     edges[cnt].to = to;
152     edges[cnt].next = head[from];
153     head[from] = cnt;
154 }
155 void bfs()
156 {
157     for(int i = 0; i <= n; i++)
158     {
159         dep[i] = 0;
160         gap[i] = 0;
161     }
162     dep[t] = 1;
163     queue<int> q;
164     q.push(t);
165     while(!q.empty())
166     {
167         int x = q.front();
168         q.pop();
169         gap[dep[x]]++;
170         for(int i = head[x]; i; i = edges[i].next)
171         {
172             int to = edges[i].to;
173             if(edges[i^1].w && !dep[to])
174             {
175                 dep[to] = dep[x] + 1;
176                 q.push(to);
177             }
178         }
179     }
```

```

179     }
180 }
181 ll augment()
182 {
183     ll v = t, flow = INF;
184     while(v != s)
185     {
186         int u = pre[v];
187         flow = min(flow, edges[u].w);
188         v = edges[u].from;
189     }
190     v = t;
191     while(v != s)
192     {
193         int u = pre[v];
194         edges[u].w -= flow;
195         edges[u^1].w += flow;
196         v = edges[u].from;
197     }
198     return flow;
199 }
200
201 void ISAP()
202 {
203     bfs();
204     ll flow = 0;
205     int u = s;
206     for(int i = 1; i <= n; i++) now[i] = head[i];
207     while(dep[s] <= n)
208     {
209         if(u == t)
210         {
211             flow += augment();
212             u = s;
213         }
214         bool ok = 0;
215         for(int i = now[u]; i; i = edges[i].next)
216         {
217             int v = edges[i].to;
218             if(edges[i].w && dep[v] + 1 == dep[u])
219             {
220                 ok = 1;
221                 pre[v] = i;
222                 now[u] = i;
223                 u = v;
224                 break;
225             }
226         }
227         if(!ok)
228         {
229             gap[dep[u]]--;
230             if(!gap[dep[u]]) break;
231             int mi = n + 10;
232             for(int i = head[u]; i; i = edges[i].next)
233             {
234                 int v = edges[i].to;

```

```

235         if(dep[v] < mi && edges[i].w)
236             mi = dep[v];
237     }
238     dep[u] = mi + 1;
239     gap[dep[u]]++;
240     now[u] = head[u];
241     if(u != s)
242         u = edges[pre[u]].from;
243     }
244 }
245 cout << flow << "\n";
246 }
247 void solve()
248 {
249     cin >> n >> m >> s >> t;
250     for(int i = 1; i <= m; i++)
251     {
252         int a, b;
253         ll c;
254         cin >> a >> b >> c;
255         add(a, b, c);
256         add(b, a, 0);
257     }
258     ISAP();
259 }
260
261 // 最大闭合子图的权值等于所有正权点之和减最小割

```

2.21 边双联通分量

```

1 // tarjan求边双
2 ll n, m, u, v, cnt;
3 int dfn[100005], low[100005];
4 vector<PII> g[100005];
5 stack<int> st;
6 int mm;
7 int scc[100005];
8 int sz[100005];
9 void tarjan(int x, int las){
10     low[x] = dfn[x] = ++cnt;
11     st.push(x);
12     for (auto it: g[x]){
13         if (it.se == (las ^ 1)) continue;
14         if (!dfn[it.fi]){
15             tarjan(it.fi, it.se);
16             low[x] = min(low[x], low[it.fi]);
17         }else low[x] = min(low[x], dfn[it.fi]);
18     }
19     if (dfn[x] == low[x]){
20         vector<int> vec;
21         scc[x] = ++mm;
22         sz[mm]++;
23         while (st.top() != x){
24             scc[st.top()] = mm;

```



```

25         sz[mm]++;
26         st.pop();
27     }
28     st.pop();
29 }
30 }
31 for(int i = 1; i <= m; i++)
32 {
33     int a, b;
34     cin >> a >> b;
35     g[a].pb({b, i << 1});
36     g[b].pb({a, i << 1|1});
37 }
38 for(int i = 1; i <= n; i++)
39 {
40     if(!dfn[i])
41         tarjan(i, 0);
42 }

```

2.22 链式前向星

```

1 struct edge
2 {
3     int to, w, next;
4 } edges[m];
5 int head[n], cnt;
6 void add(int from, int to, int w)
7 {
8     edges[++cnt].w = w;
9     edges[cnt].to = to;
10    edges[cnt].next = head[from];
11    head[from] = cnt;
12 }

```

3 字符串

3.1 AC 自动机

```

1 //AC自动机
2 // 求有多少个不同的模式串在文本串中出现过，两个模式串不同当且仅当题面编号不同0(文本串长
   度 + 模式串总长度)
3 int nex[1000005][26], cnt;
4 int exist[1000005];
5 int fail[1000005];
6 int add()
7 {
8     ++cnt;
9     memset(nex[cnt], 0, sizeof(nex[cnt]));
10    exist[cnt] = 0;
11    fail[cnt] = 0;
12    return cnt;

```

```
13 }
14 void init()
15 {
16     cnt = -1;
17     add();
18 }
19 void insert(string s)
20 {
21     int cur = 0;
22     for (auto it : s)
23     {
24         if(!nex[cur][it - 'a'])
25             nex[cur][it - 'a'] = add();
26         cur = nex[cur][it - 'a'];
27     }
28     exist[cur]++;
29 }
30 void getfail()
31 {
32     queue<int> q;
33     for(int i = 0; i < 26; i++)
34     {
35         if(nex[0][i])
36         {
37             fail[nex[0][i]] = 0;
38             q.push(nex[0][i]);
39         }
40     }
41     while(!q.empty())
42     {
43         int now = q.front();
44         q.pop();
45         for(int i = 0; i < 26; i++)
46         {
47             if(nex[now][i])
48             {
49                 fail[nex[now][i]] = nex[fail[now]][i];
50                 q.push(nex[now][i]);
51             }
52             else
53                 nex[now][i] = nex[fail[now]][i];
54         }
55     }
56 }
57 int query(string s)
58 {
59     int now = 0;
60     int ans = 0;
61     for(int i = 0; i < s.size(); i++)
62     {
63         now = nex[now][s[i] - 'a'];
64         for(int j = now; j && exist[j] != -1; j = fail[j]) // exist打标记, trie上走过的点
            不再走
65     {
66         ans += exist[j];
67         exist[j] = -1;
68     }
```

```
68     }
69 }
70 return ans;
71 }
72 void solve()
73 {
74     int n;
75     cin >> n;
76     for(int i = 1; i <= n; i++)
77     {
78         string s;
79         cin >> s;
80         insert(s);
81     }
82     getfail();
83     string s;
84     cin >> s;
85     cout << query(s);
86 }
87 }
88
89
90
91 // AC自动机求每个模式串在文本串中出现次数 O(文本串长度 + 模式串总长度)
92 int nex[200005][26], cnt;
93 int exist[200005];
94 int fail[200005];
95 int in[200005];
96 int dp[200005];
97 int ans[200005];
98 int add()
99 {
100     ++cnt;
101     memset(nex[cnt], 0, sizeof(nex[cnt]));
102     exist[cnt] = 0;
103     fail[cnt] = 0;
104     return cnt;
105 }
106 void init()
107 {
108     cnt = -1;
109     add();
110 }
111 void insert(string s, int i)
112 {
113     int cur = 0;
114     for (auto it : s)
115     {
116         if(!nex[cur][it - 'a'])
117             nex[cur][it - 'a'] = add();
118         cur = nex[cur][it - 'a'];
119     }
120     exist[cur] = i;
121 }
122 void getfail()
123 {
```

```

124     queue<int> q;
125     for(int i = 0; i < 26; i++)
126     {
127         if(nex[0][i])
128         {
129             fail[nex[0][i]] = 0;
130             in[fail[nex[0][i]]]++;
131             q.push(nex[0][i]);
132         }
133     }
134     while(!q.empty())
135     {
136         int now = q.front();
137         q.pop();
138         for(int i = 0; i < 26; i++)
139         {
140             if(nex[now][i])
141             {
142                 fail[nex[now][i]] = nex[fail[now]][i];
143                 in[fail[nex[now][i]]]++;
144                 q.push(nex[now][i]);
145             }
146             else
147                 nex[now][i] = nex[fail[now]][i];
148         }
149     }
150 }
151
152 void query(string s)
153 {
154     int now = 0;
155     for(int i = 0; i < s.size(); i++)
156     {
157         now = nex[now][s[i] - 'a'];
158         dp[now]++;
159     }
160 }
161 map<string, int> mp;
162 string s[200005];
163 int pos = 0;
164 void topu()
165 {
166     queue<int> q;
167     for(int i = 1; i <= cnt; i++)
168     {
169         if(!in[i])
170             q.push(i);
171     }
172     while(!q.empty())
173     {
174         int x = q.front();
175         q.pop();
176         ans[exist[x]] = dp[x];
177         int to = fail[x];
178         in[to]--;
179         dp[to] += dp[x];

```

```

180         if(!in[to])
181             q.push(to);
182     }
183 }
184 void solve()
185 {
186     init();
187     int n;
188     cin >> n;
189     for(int i = 1; i <= n; i++)
190     {
191         cin >> s[i];
192         if(!mp.count(s[i]))
193             mp[s[i]] = ++pos;
194         insert(s[i], mp[s[i]]);
195     }
196     getfail();
197     string s1;
198     cin >> s1;
199     query(s1);
200     topu();
201     for(int i = 1; i <= n; i++)
202         cout << ans[mp[s[i]]] << "\n";
203 }

```

3.2 hash

```

1 ll get(int l, int r)
2 {
3     return h[r] - h[l-1] * p[r-l+1];
4 }
5 p[0] = 1;
6 for(int i = 1; i <= n; i++)
7 {
8     h[i] = h[i-1]*base+s[i];
9     p[i] = p[i-1]*base;
10 }

```

3.3 kmp

```

1 // kmp 在s中找p O(n+m)
2 void getnext() // next[i] : p[0] 到 p[i-1] 中 最长前后缀交集(前后缀不包括整个串)
3 {
4     next[0] = 0, next[1] = 0;
5     for(int i = 1; i < p.size(); i++)
6     {
7         int j = next[i];
8         while(j && p[i] != p[j])
9             j = next[j];
10        if(p[i] == p[j]) next[i+1] = j+1;
11        else next[i+1] = 0;
12    }

```

```

13 }
14 void kmp()
15 {
16     int j = 0;
17     for(int i = 0; i < s.size(); i++)
18     {
19         while(j && s[i] != p[j])
20             j = next[j];
21         if(s[i] == p[j])
22             j++;
23         if(j == p.size())
24         {
25             //匹配成功
26         }
27     }
28 }

```

3.4 字典树

```

1  //(空间开: 总字符个数 * 字符集大小)
2  int nex[MAXN][26], cnt;
3  int exist[MAXN];
4  void init()
5  {
6      cnt = -1;
7      add();
8  }
9  int add()
10 {
11     ++cnt;
12     memset(nex[cnt], 0, sizeof(nex[cnt]));
13     exist[cnt] = 0;
14     return cnt;
15 }
16 void insert(string s)
17 {
18     int cur = 0;
19     for (auto it : s)
20     {
21         if(!nex[cur][it - 'a'])
22             nex[cur][it - 'a'] = add();
23         cur = nex[cur][it - 'a'];
24     }
25     // exist[cur] = 1;
26 }
27 ll find_pre(string s)
28 {
29     int cur = 0;
30     for (auto it : s)
31     {
32         if(!nex[cur][it - 'a'])
33             return false;
34         cur = nex[cur][it - 'a'];
35     }

```

```

36     return exist[cur];
37 }

```

3.5 马拉车

```

1  // manacher
2  // P[i] 是以字符s[i]为中心字符的最长回文串的半径,最大的p[i]-1就是最长回文串长度,这个最长
   回文串在原字符串的开头位置是(i-p[i])/2;
3  int p[n << 1];
4  string s;
5  string s1 = "$#";
6  void manacher()
7  {
8      int r = 0, c;
9      for(int i = 0; i < s1.size(); i++)
10     {
11         if(i < r) p[i] = min(p[(c << 1) - i], p[c] + c - i);
12         else p[i] = 1;
13         while(s1[i + p[i]] == s1[i - p[i]]) p[i]++;
14         if(p[i] + i > r)
15         {
16             r = p[i] + i;
17             c = i;
18         }
19     }
20 }
21
22 void solve()
23 {
24     cin >> s;
25     for(auto it : s)
26     {
27         s1 += it;
28         s1 += '#';
29     }
30     s1 += '&';
31     manacher();
32     int ans = 0;
33     for(int i = 0; i < s1.size(); i++)
34         ans = max(ans, p[i]-1);
35     cout << ans;
36
37 }

```

4 数学

4.1 MillerRabin

```

1  //MillerRabin O(logn) 判断一个数是不是素数
2  srand((unsigned)time(NULL));
3  ll ksm(ll a, ll b, ll m)

```

```

4 {
5     a %= m;
6     ll ans = 1;
7     while(b)
8     {
9         if(b&1) ans = (LL)ans * a % m;
10        a = (LL)a * a % m;
11        b = b >> 1;
12    }
13    return ans;
14 }
15 bool Miller_Rabin(ll p) { // 判断素数
16     if (p < 2) return 0;
17     if (p == 2) return 1;
18     if (p == 3) return 1;
19     ll d = p - 1, r = 0;
20     while (!(d & 1)) ++r, d >>= 1; // 将d处理为奇数
21     for (ll k = 0; k < 10; ++k) {
22         ll a = rand() % (p - 2) + 2;
23         ll x = ksm(a, d, p);
24         if (x == 1 || x == p - 1) continue;
25         for (int i = 0; i < r - 1; ++i) {
26             x = (LL)x * x % p;
27             if (x == p - 1) break;
28         }
29         if (x != p - 1) return 0;
30     }
31     return 1;
32 }

```

4.2 Miller_Rabin + Pollard_Rho 分解质因子

```

1 // Miller_Rabin + Pollard_Rho 分解质因子
2 void fac(ll x) {
3     if(x < 2)
4         return;
5     if(Miller_Rabin(x))
6     {
7         mp[x]++;
8         return;
9     }
10    ll p = Pollard_Rho(x);
11    fac(p), fac(x/p);
12 }

```

4.3 Pollard_Rho

```

1 //Pollard_Rho 快速找到x的一个因子(非1非自身)  $O(n^{1/4})$ 
2 srand((unsigned)time(NULL));
3 ll Pollard_Rho(ll x) {
4     ll s = 0, t = 0;
5     ll c = (ll)rand() % (x - 1) + 1;

```



```

6  ll step = 0, goal = 1;
7  ll val = 1;
8  for (goal = 1;; goal <= 1, s = t, val = 1) { // 倍增优化
9      for (step = 1; step <= goal; ++step) {
10         t = ((LL)t * t + c) % x;
11         val = (LL)val * abs(t - s) % x;
12         if ((step % 127) == 0) {
13             ll d = gcd(val, x);
14             if (d > 1) return d;
15         }
16     }
17     ll d = gcd(val, x);
18     if (d > 1) return d;
19 }
20 }

```

4.4 中国剩余定理

```

1  // 保证模数两两互质，这个函数返回的是符合方程组的最小正数解，一般要求的正是这个。在  $a_1 * a_2 * a_3 * \dots * a_n$  内存在唯一解
2  ll CRT(int k) // k 是数组长度
3  {
4      ll n = 1, ans = 0;
5      for(int i=0;i<k;i++) n = n * mod[i];
6      for(int i=0;i<k;i++)
7      {
8          ll m = n/mod[i];
9          ll b,y;
10         exgcd(m,mod[i],b,y);
11         ans = (ans + r[i] * m * b % n) % n;
12     }
13     return (ans % n + n) % n;
14 }

```

4.5 区间筛

```

1  //区间筛 筛[a,b)内的质数
2  int primes0[1000005],primes1[1000005];
3  ll prime[1000005];
4  int cnt = 0;
5  void segment_sieve(ll a,ll b)
6  {
7      for(ll i = 0;i * i < b;i++) primes0[i] = 1;
8      for(ll i = 0;i < b-a;i++) primes1[i] = 1;
9      for(ll i = 2;i * i < b;i++)
10     {
11         if(primes0[i])
12         {
13             for(ll j = 2 * i;j * j < b;j += i) primes0[j] = 0;
14             for(ll j = max(2ll,(a + i - 1) / i) * i;j < b;j += i) primes1[j-a] = 0;
15         }
16     }

```

```

17     for(ll i = 0; i < b-a; i++)
18     {
19         if(primes1[i]) prime[++cnt] = i+a;
20     }
21 }

```

4.6 卢卡斯定理

```

1  // 需要先预处理出fact[], 即阶乘
2  ll inv(ll a, ll p)
3  {
4      return qpow(a, p-2);
5  }
6  ll C(ll m, ll n, ll p)
7  {
8      return m < n ? 0 : fact[m] * inv(fact[n], p) % p * inv(fact[m - n], p) % p;
9  }
10 ll lucas(ll m, ll n, ll p)
11 {
12     return n == 0 ? 1 % p : lucas(m / p, n / p, p) * C(m % p, n % p, p) % p;
13 }

```

4.7 因子个数 $O(n)$

```

1  int p[n], g[n], cnt; // p为质数集, g[i]为i的因子个数
2  bool inp[n];
3  void init(int n) {
4      g[1] = 1;
5      for (int i = 2; i <= n; ++i) {
6          if (!inp[i]) {
7              p[++cnt] = i;
8              g[i] = 2;
9          }
10         for (int j = 1; j <= cnt && i * p[j] <= n; ++j) {
11             inp[i * p[j]] = true;
12             if (i % p[j] == 0) {
13                 int t = i, o = 0;
14                 while (t % p[j] == 0) {t /= p[j]; ++o;}
15                 g[i * p[j]] = g[i] / (o + 1) * (o + 2);
16                 break;
17             } else {
18                 g[i * p[j]] = g[i] * 2;
19             }
20         }
21     }
22 }

```

4.8 因子个数 $O(n \log n)$

```

1 // g[i] 代表i的因子个数
2 for (int i = 1; i <= n; i++) {
3     for (int j = i; j <= n; j += i) {
4         g[j]++;
5     }
6 }

```

4.9 因子和 $O(n)$

```

1 vector<int> prim;
2 int vis[n];
3 int ans[n], query[n], num[n]; // ans[i] 为 i 的因子和
4 void euler()//筛法求因子和
5 {
6     ans[1] = 1;
7     for(int i = 2; i < n; i++)
8     {
9         if(!vis[i])
10        {
11            prim.push_back(i);
12            num[i] = ans[i] = i + 1;
13        }
14        for(int j = 0 ; j < prim.size() && prim[j] <= n / i; j++)
15        {
16            vis[i*prim[j]] = 1;
17            if(i%prim[j]==0)
18            {
19                num[prim[j] * i] = num[i] * prim[j] + 1;
20                ans[prim[j] * i] = ans[i]/num[i]*num[prim[j] * i];
21                break;
22            }
23            else
24            {
25                ans[prim[j]*i] = ans[i]*(1 + prim[j]);
26                num[prim[j]*i] = prim[j] + 1;
27            }
28        }
29    }
30 }

```

4.10 因子和 $O(n\log n)$

```

1 // g[i]代表i的因子和
2 for(int i = 1; i <= n; i++)
3 {
4     for(int j = i; j <= n; j+=i)
5         g[j] += i;
6 }

```

4.11 威尔逊定理

```

1 // 威尔逊定理
2 // m 为 4 时 ,  $(m - 1)! = 2 \pmod{m}$ 
3 // m 为 质数 时 ,  $(m - 1)! = m-1 \pmod{m}$ 
4 // m 不为 质数 时 ,  $(m - 1)! = 0 \pmod{m}$ 

```

4.12 扩展欧几里得

```

1 // a > b;
2 //  $ax_0 + by_0 = \gcd(a,b)$ ;
3 // 如果参数为负数,同余方程是在模意义下的,取参数在模意义下的最小正数即可。
4 //  $ax + by = c$  的一个特解是  $x_1 = x_0*c/\gcd(a,b)$   $y_1 = y_0*c/\gcd(a,b)$ ;
5 //  $dx = b/\gcd(a,b)$   $dy = a/\gcd(a,b)$ ;
6 // 通解是  $x = x_1 + s*dx$   $y = y_1 - s*dy$ ;
7 // 考虑x,y均为正数  $L = \text{ceil}(-x_1+1,dx) \leq s \leq \text{floor}(y_1-1,dy) = R$ ;
8 // 如果  $L > R$ ,则不存在正整数解
9 // 否则,存在正整数解,解的个数是  $R - L + 1$ ,  $s = L$  时可以得到  $x_{\min}, y_{\max}$ ,  $s = R$  时可以得到  $x_{\max}, y_{\min}$ ; (取模)
10 // 判断一个数k是否在gcd过程中出现  $\text{if}(a \% b == k \% b \ \&\& \ k \leq a)$ 
11 // x的最小正整数解  $x = (x_1 \% dx + dx)\%dx$ ;
12 // x的最大负整数解  $x = \text{模数} - x \text{的最小正整数解}$ 
13 ll exgcd(ll a,ll b,ll &x0,ll &y0)
14 {
15     if(b == 0)
16     {
17         x0 = 1;
18         y0 = 0;
19         return a;
20     }
21     ll d = exgcd(b,a%b,y0,x0);
22     y0 -= (a / b) * x0;
23     return d;
24 } // 求出 x0,y0
25
26 ll inv(ll a,ll p)
27 {
28     ll x,y;
29     if(exgcd(a,p,x,y) != 1)
30         return -1;
31     return (x % p + p) % p;
32 }

```

4.13 整除分块

```

1 // 除数范围1-n 被除数 : k
2 //  $O(\sqrt{n})$ 
3 for (int l = 1,r;l <= n;l = r + 1)
4 {
5     r = k / (k / l);
6 }

```

4.14 欧拉函数

```

1 // 欧拉函数 (小于等于n的正整数中与n互质的个数)
2 int phi(int n) // 求单个点的欧拉函数 O(sqrt(n))
3 {
4     int res=n;
5     for(int i=2;i*i<=n;i++)
6     {
7         if(n%i==0)
8         {
9             res = res * (i - 1) / i;
10            while(n%i==0) n/=i;
11        }
12    }
13    if(n>1) res = res * (n - 1) / n;
14    return res;
15 }
16 void phi() // 求1-n的欧拉函数 O(n)
17 {
18     phi[1] = 1;
19     int cnt = 0;
20     for(ll i = 2;i <= n;i++)
21     {
22         if(!vis[i])
23         {
24             vis[i] = 1;
25             prime[cnt++] = i;
26             phi[i] = i-1;
27         }
28         for(int j = 0;j < cnt;j++)
29         {
30             if(i * prime[j] > n) break;
31             vis[i * prime[j]] = 1;
32             if(i % prime[j] == 0)
33             {
34                 phi[i * prime[j]] = phi[i] * prime[j];
35                 break;
36             }
37             phi[i * prime[j]] = phi[i] * phi[prime[j]];
38         }
39     }
40 }

```

4.15 欧拉筛

```

1 //欧拉筛
2 // 每个合数只被他的最小质因子筛掉
3 // (快速找到每个数最小质因子)
4 bool isnp[n];
5 vector<ll> primes;
6 void init(ll n)
7 {
8     isnp[1] = 1;
9     for (ll i = 2; i <= n; i++)

```

```

10 {
11     if (!isnp[i])
12         primes.push_back(i);
13     for (ll it : primes)
14     {
15         if (it * i > n)
16             break;
17         isnp[it * i] = 1;
18         if (i % it == 0)
19             break;
20     }
21 }
22 }

```

4.16 求 x 的所有因子

```

1 // 求x的所有因子 时间复杂度  $\max(\sqrt{x}, x \text{的因子数})$ 
2 map<int, int> mp;
3 for(int i = 2; i * i <= x; i++)
4 {
5     if(x % i == 0)
6     {
7         while(x % i == 0)
8         {
9             mp[i]++;
10            x /= i;
11        }
12    }
13 }
14 if(x > 1)
15     mp[x]++;
16 vector<PII> v;
17 for(auto it : mp)
18     v.pb({it.fi, it.se});
19 vector<int> ans;
20 void dfs(int p, int x)
21 {
22     if(p == v.size())
23     {
24         ans.pb(x);
25         return;
26     }
27     for(int i = 0; i <= v[p].se; i++)
28     {
29         dfs(p+1, x);
30         x *= it.fi;
31     }
32 }
33 dfs(0, 1);

```

4.17 求区间内和 k 互质数的个数

```

1 //求区间L R 内与k互质的数的个数
2 vector<int> primes;
3 int num = 0;
4 int sum = 0;
5 for(ll i = 2;i * i <= n;i++)
6 {
7     if(n%i == 0)
8     {
9         primes.pb(i);
10        while(n%i==0)
11            n/=i;
12    }
13 }
14 if(n > 1)
15 primes.pb(n);
16
17 void dfs(int pos,int mul,int len)
18 {
19     if(pos == primes.size())
20     {
21         if(len)
22         {
23             if(len&1)
24                 sum += num/mul;
25             else
26                 sum -= num/mul;
27         }
28         return;
29     }
30     dfs(pos+1,mul,len);
31     dfs(pos+1,mul*primes[pos],len+1);
32 }
33 int ask(int r,int k)
34 {
35     num = r; sum = 0;
36     dfs(0,1,0);
37     return r - sum;
38 }

```

4.18 矩阵

```

1 // 定义矩阵
2 struct matrix
3 {
4     ll x[105][105];
5     matrix(){memset(x,0,sizeof(x));}
6 }
7 // 矩阵乘法 ()
8 matrix multiply(matrix &a,matrix &b)
9 {
10    matrix c;
11    for(int i = 1;i <= n;i++)
12    {
13        for(int j = 1;j <= n;j++)

```

```

14     {
15         for(int k = 1;k <= n;k++)
16             c.x[i][j] += a.x[i][k] * b.x[k][j];
17     }
18 }
19 return c;
20 }
21 // 矩阵快速幂
22 matrix mksm(matrix &a,ll m)
23 {
24     matrix res;
25     for(int i = 1;i <= n;i++)
26         res.x[i][i] = 1;
27     while(m)
28     {
29         if(m&1) res = multiply(res,a);
30         a = multiply(a,a);
31         m = m >> 1;
32     }
33     return ans;
34 }

```

4.19 等差乘等比求和

```

1 形如这样: (a*n+b)*ksm(q,n-1); q != 0 && q != 1
2  s = (An + B)*ksm(q,n) - B;
3  A = a * inv(q-1);
4  B = (b - A) * inv(q-1);

```

4.20 等差数列求和

```

1  s = n*a1 + n*(n-1)d * inv(2);

```

4.21 等比数列求和

```

1  s = a1 * (ksm(q,n) - 1) * inv(q - 1);

```

4.22 线性基

```

1  // 线性基
2  int cnt = 0; //如果线性基大小小于原数组大小, 说明能构造出来异或为0的子集
3  void ins(ll x) //构造线性基
4  {
5      for(int i = 60;i >= 0;i--)
6      {
7          if(x >> i & 1)
8          {
9              if(!p[i])

```



```

10         {
11             p[i] = x;
12             cnt++;
13             break;
14         }
15         else
16             x ^= p[i];
17     }
18 }
19 }
20
21 ll ask(ll x) //查询元素x能否被异或出来
22 {
23     for(int i = 60; i >= 0; i--)
24         if(x >> i & 1) x ^= p[i];
25     return x == 0;
26 }
27 ll askma() // 查询异或最大值
28 {
29     ll ans = 0;
30     for(int i = 60; i >= 0; i--)
31         if((ans ^ p[i]) > ans) ans ^= p[i];
32     return ans;
33 }
34 ll askmi() // 查询异或最小值(记得特判0)
35 {
36     if(zero) return 0;
37     for(int i = 0; i <= 60; i++)
38     {
39         if(p[i])
40             return p[i];
41     }
42 }
43 }
44 void rebuild() //重新构造, 方便查询异或第k小
45 {
46     cnt = 0;
47     for(int i = 60; i >= 0; i--)
48     {
49         for(int j = i-1; j >= 0; j--)
50         {
51             if(p[i] >> j & 1)
52                 p[i] ^= p[j];
53         }
54     }
55     for(int i = 0; i <= 60; i++)
56     {
57         if(p[i])
58             d[cnt++] = p[i];
59     }
60 }
61 ll kth(ll k) //找异或第k小(记得特判0)
62 {
63     if(k >= (1ll << cnt)) return -1; // 找不到
64     ll ans = 0;
65     for(int i = 60; i >= 0; i--)

```

```

66     if(k >> i & 1) ans ^= d[i];
67     return ans;
68 }

```

4.23 线性求逆元

```

1 inv[1] = 1;
2 for(int i = 2; i <= n; ++ i)
3     inv[i] = (ll)(p - p / i) * inv[p % i] % p;

```

4.24 组合数

```

1 fac[0] = 1;
2 for (int i = 1; i <= 1000; i++)
3     fac[i] = (fac[i-1] * i) % mod;
4 ifac[1000] = ksm(fac[1000], mod - 2);
5 for (int i = 1000; i >= 1; i--)
6     ifac[i - 1] = ifac[i] * i % mod;
7 ll C(int n, int m)
8 {
9     if (n < m || m < 0) return 0;
10    return fac[n] * ifac[m] % mod * ifac[n - m] % mod;
11 }
12
13 ll A(int n, int m)
14 {
15     if (n < m || m < 0) return 0;
16     return fac[n] * ifac[n - m] % mod;
17 }

```

4.25 莫比乌斯函数

```

1 void mobius() // 求1-n的莫比乌斯函数 O(n)
2 {
3     int cnt = 0;
4     vis[1] = 1;
5     mob[1] = 1;
6     for(ll i = 2; i <= n; i++)
7     {
8         if(!vis[i]) prime[cnt++] = i, mob[i] = -1;
9         for(int j = 0; j < cnt && prime[j] * i <= n; j++)
10        {
11            vis[prime[j] * i] = 1;
12            mob[i * prime[j]] = (i % prime[j] ? -mob[i] : 0);
13            if(i % prime[j] == 0) break;
14        }
15    }
16 }

```

4.26 费马小定理

```

1 //费马小定理
2 //若p为素数, gcd(a,p) == 1, 则  $a^{(p-1)} \equiv 1 \pmod{p}$ 
3 //费马小定理降幂
4  $a^x \pmod{p} \equiv a^{(x \% (p-1))} \pmod{p}$ 

```

4.27 降幂

```

1 //费马小定理降幂
2  $a^x \pmod{p} \equiv a^{(x \% (p-1))} \pmod{p}$ 

```

4.28 预处理 log2

```

1 for (int i = 1; i <= n; i++)
2 {
3     lg[i] = lg[i-1] + (1 << (lg[i-1] + 1) == i);
4 }

```

5 数据结构

5.1 LCA

```

1 void dfs(int x, int father) // 求深度
2 {
3     fa[x][0] = father;
4     dep[x] = dep[father] + 1;
5     for (int i = 1; i <= 20; i++)
6         fa[x][i] = fa[fa[x][i-1]][i-1];
7     for (auto it : g[x])
8     {
9         int to = it;
10        if (to != father)
11            dfs(to, x);
12    }
13 }
14 int lca(int a, int b)
15 {
16     if (dep[a] > dep[b])
17         swap(a, b);
18     for (int i = 20; i >= 0; i--)
19     {
20         if (dep[fa[b][i]] >= dep[a])
21             b = fa[b][i];
22     }
23     if (a == b)
24         return a;
25     for (int k = 20; k >= 0; k--)

```

```

26     if (fa[a][k] != fa[b][k])
27         a = fa[a][k], b = fa[b][k];
28     return fa[a][0];
29 }

```

5.2 ST 表

```

1  for (int i = 1; i <= n; i++)
2  {
3      int x;
4      cin >> x;
5      ma[i][0] = x;
6  }
7  for (int j = 1; j <= 20; j++)
8  {
9      for (int i = 1; i + (1 << j) - 1 <= n; i++)
10         ma[i][j] = max(ma[i][j-1], ma[i + (1 << (j - 1))][j-1]);
11 }
12 for (i = 0; i < m; i++)
13 {
14     int l, r;
15     cin >> l >> r;
16     int s = lg[r - l + 1];
17     int ma1 = max(ma[l][s], ma[r - (1 << s) + 1][s]);
18 }

```

5.3 cdq 分治求三维偏序

```

1  struct node
2  {
3      ll x, y, z, w, ans;
4  } a[MAXN], b[MAXN];
5  bool cmpx(node a, node b)
6  {
7      if (a.x != b.x)
8          return a.x < b.x;
9      else
10     {
11         if (a.y != b.y)
12             return a.y < b.y;
13         return a.z < b.z;
14     }
15 }
16 bool cmpy(node a, node b)
17 {
18     if (a.y != b.y)
19         return a.y < b.y;
20     return a.z < b.z;
21 }
22 ll t[MAXN];
23 int lowbit(int x)
24 {

```

```

25     return x & (-x);
26 }
27 void add(int pos,int x)
28 {
29     for(int i = pos;i < MAXN;i += lowbit(i))
30         t[i] += x;
31 }
32 ll ask(int pos)
33 {
34     ll ans = 0;
35     for(int i = pos;i; i -= lowbit(i))
36         ans += t[i];
37     return ans;
38 }
39 void cdq(int l,int r)
40 {
41     if(l == r) return;
42     int mid = l + r >> 1;
43     cdq(l,mid),cdq(mid + 1,r);
44     sort(b+l,b+l+mid,cmpy);
45     sort(b+l+mid,b+l+r,cmpy);
46     int i = l;
47     for(int j = mid + 1;j <= r;j++)
48     {
49         while(b[i].y <= b[j].y && i <= mid)
50             add(b[i].z,b[i].w),i++;
51         b[j].ans += ask(b[j].z);
52     }
53     for(int j = l;j < i;j++)
54         add(b[j].z,-b[j].w);
55 }
56 ll cnt[MAXN];
57 void solve()
58 {
59     int n,k;
60     cin >> n >> k;
61     for(int i = 1;i <= n;i++)
62         cin >> a[i].x >> a[i].y >> a[i].z;
63     sort(a+1,a+1+n,cmpx);
64     int nn = 0,c = 0;
65     for(int i = 1;i <= n;i++) // 去重
66     {
67         c++;
68         if(a[i].x != a[i+1].x || a[i].y != a[i+1].y || a[i].z != a[i+1].z)
69             b[++nn] = a[i],b[nn].w = c,c = 0;
70     }
71     cdq(1,nn);
72     for(int i = 1;i <= nn;i++)
73         cnt[b[i].ans + b[i].w - 1] += b[i].w;
74     for(int i = 0;i < n;i++)
75         cout << cnt[i] << '\n';
76 }
77 }

```

5.4 dsu on tree

```

1 //dsu on tree  $O(n \log n)$  (cf 600E)
2 map<int,int> dfs(int x,int fa)
3 {
4     map<int,int> mp;
5     mp[c[x]] = 1;
6     mx[x] = 1;
7     ans[x] = c[x];
8     for(auto to : g[x])
9     {
10         if(to == fa)
11             continue;
12         map<int,int> mp2 = dfs(to,x);
13         if(mp.size() < mp2.size())
14         {
15             mx[x] = mx[to];
16             ans[x] = ans[to];
17             swap(mp,mp2);
18         }
19         for(auto it : mp2)
20         {
21             mp[it.fi] += it.se;
22             if(mp[it.fi] > mx[x])
23                 ans[x] = it.fi, mx[x] = mp[it.fi];
24             else if(mp[it.fi] == mx[x])
25                 ans[x] += it.fi;
26         }
27     }
28     return mp;
29 }

```

5.5 三分

```

1 ll l = 0, r = 1e18;
2 while (l < r)
3 {
4     ll lmid = l + (r-l)/3, rmid = r - (r-l)/3;
5     if(f(rmid) < f(lmid))
6         l = lmid + 1;
7     else
8         r = rmid - 1;
9 }
10 printf("%.10lf", f(l));

```

5.6 主席树前 k 大之和

```

1 int n,m;
2 ll a[MAXN];
3 ll b[MAXN];
4 int root[MAXN];
5 int cnt;

```

```

6  int nn;
7  struct node
8  {
9      ll l,r,num,sum,s,val;
10 }t[MAXN << 5];
11 int update(int pre,int x,ll v,int cl = 1,int cr = nn)
12 {
13     int rt = ++cnt;
14     t[rt].l = t[pre].l;
15     t[rt].r = t[pre].r;
16     t[rt].num = t[pre].num + 1;
17     t[rt].sum = t[pre].sum + v;
18     int mid = cl + cr >> 1;
19     if(cl == cr)
20     {
21         t[rt].val = v;
22         return rt;
23     }
24     if(cl < cr)
25     {
26         if(x <= mid)
27             t[rt].l = update(t[pre].l,x,v,cl,mid);
28         else
29             t[rt].r = update(t[pre].r,x,v,mid + 1,cr);
30     }
31     return rt;
32 }
33 ll ask(int l,int r,int k,int cl = 1,int cr = nn)
34 {
35     if(cl == cr)
36         return t[r].val * k;
37     int x = t[t[r].r].num - t[t[l].r].num;
38     int mid = cl + cr >> 1;
39     if(x >= k)
40         return ask(t[l].r,t[r].r,k,mid+1,cr);
41     else
42         return t[t[r].r].sum - t[t[l].r].sum + ask(t[l].l,t[r].l,k-x,cl,mid);
43 }
44 void solve()
45 {
46     cnt = 0;
47     cin >> n;
48     for(int i = 1;i <= n;i++)
49         cin >> a[i],b[i] = a[i];
50     sort(b+1,b+1+n);
51     nn = unique(b+1,b+1+n) - b - 1;
52     for(int i = 1;i <= n;i++)
53     {
54         int p = lower_bound(b+1,b+1+nn,a[i]) - b;
55         root[i] = update(root[i-1],p,a[i]);
56     }
57     cin >> m;
58     for(int i = 1;i <= m;i++)
59     {
60         int l,r,k;
61         cin >> l >> r >> k;

```

```

62     ll ans = ask(root[l-1],root[r],k);
63     ll tmp = r - l + 1;
64     cout << tmp * (tmp + 1) * (2 * tmp + 1) / 6 + ans << "\n";
65     // cout << ans << "\n";
66 }
67 }

```

5.7 主席树区间第 k 小

```

1  // 可持久化线段树求区间第k小 多测令cnt = 0
2  int n,m;
3  ll a[MAXN];
4  ll b[MAXN];
5  int root[MAXN];
6  int cnt;
7  int nn;
8  struct node
9  {
10     int l,r,num;
11 }t[MAXN << 5];
12 int update(int pre,int x,int cl = 1,int cr = nn)
13 {
14     int rt = ++cnt;
15     t[rt].l = t[pre].l;
16     t[rt].r = t[pre].r;
17     t[rt].num = t[pre].num + 1;
18     int mid = cl + cr >> 1;
19     if(cl < cr)
20     {
21         if(x <= mid)
22             t[rt].l = update(t[pre].l,x,cl,mid);
23         else
24             t[rt].r = update(t[pre].r,x,mid + 1,cr);
25     }
26     return rt;
27 }
28 int ask(int l,int r,int k,int cl = 1,int cr = nn)
29 {
30     if(cl == cr)
31         return cl;
32     int x = t[t[r].l].num - t[t[l].l].num;
33     int mid = cl + cr >> 1;
34     if(x >= k)
35         return ask(t[l].l,t[r].l,k,cl,mid);
36     else
37         return ask(t[l].r,t[r].r,k-x,mid + 1,cr);
38 }
39 void solve()
40 {
41     cin >> n >> m;
42     for(int i = 1;i <= n;i++)
43         cin >> a[i],b[i] = a[i];
44     sort(b+1,b+1+n);
45     nn = unique(b+1,b+1+n) - b - 1;

```



```

46     for(int i = 1; i <= n; i++)
47     {
48         int p = lower_bound(b+1, b+1+nn, a[i]) - b;
49         root[i] = update(root[i-1], p);
50     }
51     for(int i = 1; i <= m; i++)
52     {
53         int l, r, k;
54         cin >> l >> r >> k;
55         int t = ask(root[l-1], root[r], k);
56         cout << b[t] << "\n";
57     }
58 }

```

5.8 二维 ST 表

```

1  ll dp[505][505][10][10];
2  ll a[1005][1005];
3  void RMQ_init2d(int n, int m)
4  {
5      for(int i = 1; i <= n; i++)
6      {
7          for(int j = 1; j <= m; j++)
8          {
9              dp[i][j][0][0] = a[i][j];
10             }
11         }
12     int t = log((double)m) / log(2.0);
13
14     for(int i = 0; i <= t; i++)
15     {
16         for(int j = 0; j <= t; j++)
17         {
18             if(i==0&&j==0)
19                 continue;
20             for(int row = 1; row + (1<<i) - 1 <= n; row++)
21             {
22                 for(int col = 1; col + (1<<j) - 1 <= m; col++)
23                 {
24                     if(i)
25                         dp[row][col][i][j] = max(dp[row][col][i-1][j], dp[row+(1<<(i-1))][col][i-1][j]);
26                     else
27                         dp[row][col][i][j] = max(dp[row][col][i][j-1], dp[row][col+(1<<(j-1))][i][j-1]);
28                 }
29             }
30         }
31     }
32 }
33 ll RMQ_2d(int x1, int y1, int x2, int y2)
34 {
35     int k1 = log(double(x2 - x1 + 1)) / log(2.0);
36     int k2 = log(double(y2 - y1 + 1)) / log(2.0);

```

```

37     ll m1 = dp[x1][y1][k1][k2];
38     ll m2 = dp[x2 - (1<<k1) + 1][y1][k1][k2];
39     ll m3 = dp[x1][y2 - (1<<k2) + 1][k1][k2];
40     ll m4 = dp[x2 - (1<<k1) + 1][y2 - (1<<k2) + 1][k1][k2];
41     ll _max = max(max(m1,m2),max(m3,m4));
42     return _max;
43 }

```

5.9 区间异或和最值

```

1  //nlogn 01字典树优化 trie
2  int arr[MAXN],num[MAXN];
3  int nex[MAXN][2],cnt,exist[MAXN];
4  void insert(int x)
5  {
6      int cur = 0;
7      for (int i = 30;i >= 0;i--)
8      {
9          int u = x >> i & 1;
10         if(!nex[cur][u])
11             nex[cur][u] = ++cnt;
12         cur = nex[cur][u];
13         exist[cur] = 1;
14     }
15 }
16 int query(int x)
17 {
18     int ans = 0,cur = 0;
19     for (int i = 30;i >= 0;i--)
20     {
21         int u = x >> i & 1;
22         if(exist[nex[cur][!u]])
23         {
24             cur = nex[cur][!u];
25             ans = ans * 2 + 1;
26         }
27         else
28         {
29             cur = nex[cur][u];
30             ans = ans * 2;
31         }
32     }
33     return ans;
34 }
35 void solve()
36 {
37     int n;
38     cin >> n;
39     for (int i = 0;i <= cnt;i++)
40     {
41         nex[i][0] = 0;
42         nex[i][1] = 0;
43         exist[i] = 0;
44     }

```

```

45     cnt = 0;
46     for (int i = 1; i <= n; i++)
47     {
48         cin >> arr[i];
49         num[i] = num[i-1] ^ arr[i];
50     }
51     insert(0);
52     int ans = 0;
53     for (int i = 1; i <= n; i++)
54     {
55         ans = max(ans, query(num[i]));
56         insert(num[i]);
57     }
58     cout << ans << "\n";
59 }

```

5.10 单调栈

```

1  stack<int> s;
2  for(int i = 1; i <= n; i++)
3  {
4      while(!s.empty() && arr[s.top()] < arr[i])
5      {
6          r[s.top()] = i;
7          s.pop();
8      }
9      s.push(i);
10 }
11 while(!s.empty())
12 {
13     r[s.top()] = n+1;
14     s.pop();
15 }

```

5.11 单调队列找最值

```

1  //单调队列找最大值 m: 区间长度 q存储编号
2  deque<int> q;
3  for (int i = 1; i <= n; i++)
4  {
5      while(!q.empty() && i - q.front() >= m)
6          q.pop_fornt();
7      while(!q.empty() && arr[q.back()] < arr[i]) //找最小值改成>
8          q.pop_back();
9      q.push_back(i);
10     // if(i >= m)
11     //     cout << arr[q.front()];
12 }

```

5.12 对顶堆

```
1  if(a[i] > q1.top()) q2.push(a[i]);
2  else q1.push(a[i]);
3  while((int)q1.size() - (int)q2.size() > 1)
4  {
5      q2.push(q1.top());
6      q1.pop();
7  }
8  while((int)q2.size() > (int)q1.size())
9  {
10     q1.push(q2.top());
11     q2.pop();
12 }
```

5.13 带权并查集

```
1  int f[MAXN],dis[MAXN];
2  for (int i = 1;i <= n;i++)
3      f[i] = i;
4  int find(int x)
5  {
6      if(x == f[x])
7          return x;
8      int tmp = find(f[x]);
9      dis[x] = dis[f[x]] + dis[x];
10     return f[x] = tmp;
11 }
12 void merge(int a,int b,int w) // w的大小要具体分析
13 {
14     int x = find(a),y = find(b);
15     f[x] = y;
16     dis[x] = dis[b] + w - dis[a];
17 }
```

5.14 并查集

```
1  for (int i = 1;i <= n;i++)
2      f[i] = i;
3  int find(int x)
4  {
5      return x == f[x] ? x : f[x] = find(f[x]);
6  }
7  void merge(int a,int b)
8  {
9      int x = find(a),y = find(b);
10     if(x != y)
11         f[x] = y;
12 }
```

5.15 归并排序

```
1 int arr[n];
2 int brr[n];
3 void msort (int l,int r)
4 {
5     int mid = (l + r) >> 1;
6     if(l == r)
7         return;
8     else
9     {
10         msort(l,mid);
11         msort(mid+1,r);
12     }
13     int i = l;
14     int j = mid+1;
15     int t = l;
16
17     while(i <= mid && j <= r)
18     {
19         if(arr[i] > arr[j])
20         {
21             ans += mid - i + 1;
22             brr[t++] = arr[j++];
23         }
24         else
25         {
26             brr[t++] = arr[i++];
27         }
28     }
29
30     while(i <= mid)
31     {
32         brr[t++] = arr[i++];
33     }
34
35     while(j <= r)
36     {
37         brr[t++] = arr[j++];
38     }
39
40     for (int i = l;i < t;i++)
41         arr[i] = brr[i];
42     return;
43 }
```

5.16 扫描线求面积并

```
1 struct node
2 {
3     ll lx,rx,y,tp;
4 }len[MAXN];
5 ll cnt;
6 ll xx[MAXN];
7 ll tag[MAXN],t[MAXN];
8 ll m;
```

```

9  bool cmp(node a,node b)
10 {
11     return a.y < b.y;
12 };
13 void pushup(int p,int cl,int cr)
14 {
15     if(tag[p]) t[p] = xx[cr] - xx[cl];
16     else t[p] = t[p << 1] + t[p << 1 | 1];
17 }
18 void update(int l,int r,int tp,int p = 1,int cl = 1,int cr = m)
19 {
20     if(l <= cl && cr <= r)
21     {
22         tag[p] += tp;
23         pushup(p,cl,cr);
24         return;
25     }
26     if(cl + 1 == cr)
27         return;
28     int mid = cl + cr >> 1;
29     if(l <= mid) update(l,r,tp,p << 1,cl,mid);
30     if(r > mid) update(l,r,tp,p << 1 | 1,mid,cr);
31     pushup(p,cl,cr);
32 }
33 void solve()
34 {
35     int n;
36     cin >> n;
37     for(int i = 1;i <= n;i++)
38     {
39         db x1,y1,x2,y2;
40         cin >> x1 >> y1 >> x2 >> y2;
41         len[++cnt] = {x1,x2,y1,1};
42         xx[cnt] = x1;
43         len[++cnt] = {x1,x2,y2,-1};
44         xx[cnt] = x2;
45     }
46     sort(xx+1,xx+1+cnt);
47     sort(len+1,len+1+cnt,cmp);
48     m = unique(xx+1,xx+1+cnt) - xx - 1;
49     ll ans = 0;
50     for(int i = 1;i <= cnt;i++)
51     {
52         ans += t[1] * (len[i].y - len[i-1].y);
53         int L = lower_bound(xx+1,xx+1+m,len[i].lx) - xx;
54         int R = lower_bound(xx+1,xx+1+m,len[i].rx) - xx;
55         update(L,R,len[i].tp);
56     }
57     cout << ans;
58 }

```

5.17 树状数组

```
1 // 树状数组
```

```

2  int t[n];
3  // 单点修改
4  void update_n(int pos,int x)
5  {
6      for (int i = pos; i < n;i += lowbit(i))
7          t[i] += x;
8  }
9  // 前n项和
10 int ask_n(int n)
11 {
12     int ans = 0;
13     for (int i = n;i;i -= lowbit(i))
14         ans += t[i];
15     return ans;
16 }

```

5.18 树的直径

```

1  // dfs求法
2  void dfs(int x,int fa)
3  {
4      for(auto to : g[x])
5      {
6          if(to == fa) continue;
7          dis[to] = dis[x] + 1;
8          if(dis[to] > dis[p]) p = to;
9          dfs(to,x);
10     }
11 }
12 dfs(1,0);
13 dis[p] = 0;
14 dfs(p,0);
15 cout << dis[p] // 树的直径长度
16 // 如果要求出一条直径上所有的节点，则可以在第二次DFS的过程中，记录每个点的前序节点，即可
   // 从直径的一段一路向前，遍历直径上的所有节点
17 // 树形dp求法
18 int dp[MAXN];
19 void dfs(int x,int fa)
20 {
21     for(auto to : g[x])
22     {
23         if(to == fa) continue;
24         dfs(to,x);
25         ans = max(ans,dp[x] + dp[to] + 1);
26         dp[x] = max(dp[x],dp[to] + 1);
27     }
28 }

```

5.19 树的重心

```

1  // 树的重心：如果在树中选择某个节点并删除，这棵树将分为若干棵子树，统计子树节点数并记录
   // 最大值。取遍树上所有节点，使此最大值取到最小的节点被称为整个树的重心。

```

```

2 //sz : 树的大小 mss: 最大子树大小
3 vector<int> ctr // 重心
4 void dfs(int x,int father)
5 {
6     sz[x] = 1 , mss[x] = 0;
7     for (int i = head[x];i;i = edges[i].next)
8     {
9         int to = edges[i].to;
10        if(to!=father)
11        {
12            dfs(to,x);
13            sz[x] += sz[to];
14            mss[x] = max(mss[x],sz[to]);
15        }
16    }
17    mss[x] = max(mss[x],n - sz[x]);
18    if(mss[x] <= n/2)
19    {
20        ctr.push_back(x);
21    }
22 }

```

5.20 线段树

```

1 ll tree[n << 2],mark[n << 2],n,m,arr[n];
2 void build(int p = 1,int cl = 1,int cr = n)
3 {
4     if(cl == cr)
5     {
6         tree[p] = arr[cl];
7         return;
8     }
9     int mid = (cl + cr) >> 1;
10    build(p << 1,cl,mid);
11    build(p << 1 | 1,mid+1,cr);
12    tree[p] = tree[p << 1] + tree[p << 1 | 1];
13 }
14 void push_down(int p,int len)
15 {
16     mark[p << 1] += mark[p];
17     mark[p << 1 | 1] += mark[p];
18     tree[p << 1] += mark[p] * (len - len / 2);
19     tree[p << 1 | 1] += mark[p] * (len / 2);
20     mark[p] = 0;
21 }
22 void update(int l,int r,int d,int p = 1,int cl = 1,int cr = n)
23 {
24     if(cl >= l && cr <= r)
25     {
26         tree[p] += (cr - cl + 1) * d;
27         mark[p] += d;
28         return;
29     }
30     push_down(p,cr-cl+1);

```



```

31     int mid = (cl + cr) >> 1;
32     if(mid >= l) update(l,r,d,p << 1,cl,mid);
33     if(mid < r) update(l,r,d,p << 1 | 1,mid+1,cr);
34     tree[p] = tree[p << 1] + tree[p << 1 | 1];
35 }
36 ll ask(int l,int r,int p = 1,int cl = 1,int cr = n)
37 {
38     if(l <= cl && cr <= r)
39         return tree[p];
40     push_down(p,cr - cl + 1);
41     ll mid = (cr + cl) >> 1,ans = 0;
42     if(mid >= l) ans += ask(l,r,p << 1 ,cl ,mid);
43     if(mid < r) ans += ask(l,r,p << 1 | 1,mid+1,cr);
44     return ans;
45 }

```

5.21 莫队

```

1 //普通莫队 O(n * sqrt(m))
2 int belong(int x)
3 {
4     return x/b;
5 }
6 bool cmp(qry a,qry b)
7 {
8     if(belong(a.l) == belong(b.l))
9     {
10         if(belong(a.l) & 1)
11             return a.r < b.r;
12         else
13             return a.r > b.r;
14     }
15     return belong(a.l) < belong(b.l);
16 }
17 void erase(int x)
18 {
19 }
20 }
21 void insert(int x)
22 {
23 }
24 }
25 cin >> n >> m >> k;
26 b = n / (sqrt(m) + 1) + 1;
27 for (int i = 1;i <= n;i++)
28     cin >> arr[i];
29 for (int i = 1;i <= m;i++)
30 {
31     cin >> q[i].l >> q[i].r;
32     q[i].t = i;
33 }
34 sort(q+1,q+m+1,cmp);
35 int l = 1,r = 0;
36 for (int i = 1;i <= m;i++)

```

```

37 {
38     while(l < q[i].l)
39         erase(arr[l]),l++;
40     while(l > q[i].l)
41         l--,insert(arr[l]);
42     while(r < q[i].r)
43         r++,insert(arr[r]);
44     while(r > q[i].r)
45         erase(arr[r]),r--;
46     l = q[i].l , r = q[i].r;
47     Ans[q[i].t] = ans;
48 }
49 for (int i = 1;i <= m;i++)
50     cout << Ans[i] << "\n";

```

5.22 预处理 LCA

```

1 // 预处理LCA 预处理(nlogn) 查询(o1)
2 vector<int> g[MAXN];
3 int dep[MAXN],mi[MAXN][25];
4 int idx,dfn[MAXN],que[MAXN],lg[MAXN];
5 for(int i = 1;i <= idx;i++)
6     lg[i] = lg[i-1] + (1 << (lg[i-1] + 1) == i);
7 void dfs(int x,int father)
8 {
9     dfn[x] = ++idx,que[idx] = x;
10    dep[x] = dep[father] + 1;
11    for(auto it : g[x])
12    {
13        int to = it;
14        if(to != father)
15        {
16            dfs(to,x);
17            que[++idx] = x;
18        }
19    }
20 }
21 void buildst()
22 {
23     for(int i = 1;i <= idx;i++)
24         mi[i][0] = que[i];
25     for(int j = 1;j <= 20;j++)
26     {
27         for(int i = 1;i + (1 << j) - 1 <= idx;i++)
28         {
29             if(dep[mi[i][j-1]] < dep[mi[i + (1 << (j-1))][j-1]])
30                 mi[i][j] = mi[i][j-1];
31             else
32                 mi[i][j] = mi[i + (1 << (j-1))][j-1];
33         }
34     }
35 }
36 int lca(int l,int r)
37 {

```

```

38     l = dfn[l], r = dfn[r];
39     if(l > r)
40         swap(l, r);
41     int s = lg[r - l + 1];
42     if(dep[mi[l][s]] < dep[mi[r - (1 << s) + 1][s]])
43         return mi[l][s];
44     else
45         return mi[r - (1 << s) + 1][s];
46 }

```

6 杂

6.1 01 异或期望

1 对于n个非0即1的变量，他们异或的期望在n>0时为1/2，反之为0

6.2 map

```

1 //用map构造pair会复制构造造成巨大的性能损失,用move函数进行移动构造。
2 // pair<map<int,int>,int> p;
3 // map<int,int> mp;
4 // int a;
5 // {mp,a} 应该写成 {move(mp),a};
6 // pair<map<int,int>,int> tmp = pair<map<int,int>,int> dfs(...) 应写成 auto&&[] = dfs
  (...)

```

6.3 oset

```

1 template <typename T> using o_set = tree<T, null_type, less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;
2 o_set<int> s;
3 // order_of_key() 返回 o_set 中小于x的元素个数
4 // int a = s.order_of_key(x);

```

6.4 sqrt 向上取整

```

1 ll sqrt(ll n) {
2     ll lo = 0, hi = 2 * std::sqrt(n);
3     while (lo < hi) {
4         ll x = (lo + hi) / 2;
5         if (x * x >= n) {
6             hi = x;
7         } else {
8             lo = x + 1;
9         }
10    }
11    return lo;

```

```
12 }
```

6.5 上取整下取整

```
1 template <typename T, typename U>
2 T ceil(T x, U y) {
3     return (x > 0 ? (x + y - 1) / y : x / y);
4 }
5 template <typename T, typename U>
6 T floor(T x, U y) {
7     return (x > 0 ? x / y : (x - y + 1) / y);
8 }
```

6.6 二分图

```
1 //二分图最小点覆盖：给定一张二分图，求出一个最小的点集S，使得图中任意一条边都有至少一个
   端点属于S。
2 //Konig定理：二分图最小点覆盖包含的点数等于二分图最大匹配包含的边数。
3
4 //二分图最大独立集：选取尽可能多的点使得点集中所有点两两之间无边相连。
5 //定理：最大独立集 = n - 最大匹配数 (n为图的节点个数)
```

6.7 全排列

```
1 //全排列
2 do
3 {
4
5 }while(next_permutation(v.begin(),v.end()));
```

6.8 判断一个数是不是二的幂次

```
1 //判断t是不是2的幂次
2 int t;
3 if(t & (t-1) == 0)
4     cout << "YES";
5 else
6     cout << "NO"
```

6.9 博弈 1

```
1 // n堆Nim博弈 异或和为0，先手必败
2 // 阶梯博弈(Nim博弈变种) 奇数节点异或和为0，先手必败
3 // 树上阶梯博弈和阶梯博弈同理
4
5 // SG 定理
```

```

6 // SG(x) == 0 x为必败点,否则为必胜点
7 // SG(x) = mex{SG(y) | x can go to y} // y 是小状态 x 是大状态
8 // 游戏的和的SG值是它们SG值的xor

```

6.10 去重

```

1 //去重后的范围 1 到 n
2 // 数组
3 sort(a+1,a+1+n);
4 int n = unique(a+1,a+1+n) - a;
5 n--;
6 // vector
7 sort(v.begin(),v.end());
8 int n = unique(v.begin(),v.end()) - v.begin();
9 n--;

```

6.11 异或

```

1 //异或性质
2 //a - b <= a ^ b <= a + b

```

6.12 期望 1

```

1 // 如果一个事件的结果A发生的概率是P, 则一直做这件事直到第一次发生结果A的期望X是1/p(满足几何分布)
2 // 呈几何分布的期望是1/p, 方差是 (1-p)/(p*p)

```

6.13 矩形相交

```

1 //判断两个矩形是否相交
2 // 两个矩形宽高分别为w1,h1,w2,h2;
3 // 两个矩形中点间的水平距离和竖直距离分别为 w,h
4 // if(w < (w1 + w2)/2 && h < (h1 + h2) / 2) 则两个矩形相交, 否则不相交

```

6.14 规律 1

```

1 // 1^2 + 2^2 + ... + n^2 = n(n+1)(2n+1)/6
2 // 1^3 + 2^3 + ... + n^3 = (n(n+1)/2)^2

```

6.15 质数差

```

1 // 任意一个偶数x, 一定存在一对质数差为x;

```

6.16 随机模数

```

1  typedef long long ll;
2  using ll = long long;
3  template<class T>
4  constexpr T power(T a, ll b) {
5      T res {1};
6      for (; b; b /= 2, a *= a) {
7          if (b % 2) {
8              res *= a;
9          }
10     }
11     return res;
12 }
13
14 constexpr ll mul(ll a, ll b, ll p) {
15     ll res = a * b - ll(1.L * a * b / p) * p;
16     res %= p;
17     if (res < 0) {
18         res += p;
19     }
20     return res;
21 }
22
23 template<ll P>
24 struct MInt {
25     ll x;
26     constexpr MInt() : x {0} {}
27     constexpr MInt(ll x) : x {norm(x % getMod())} {}
28
29     static ll Mod;
30     constexpr static ll getMod() {
31         if (P > 0) {
32             return P;
33         } else {
34             return Mod;
35         }
36     }
37     constexpr static void setMod(ll Mod_) {
38         Mod = Mod_;
39     }
40     constexpr ll norm(ll x) const {
41         if (x < 0) {
42             x += getMod();
43         }
44         if (x >= getMod()) {
45             x -= getMod();
46         }
47         return x;
48     }
49     constexpr ll val() const {
50         return x;
51     }
52     constexpr MInt operator-() const {
53         MInt res;
54         res.x = norm(getMod() - x);

```

```

55     return res;
56 }
57 constexpr MInt inv() const {
58     return power(*this, getMod() - 2);
59 }
60 constexpr MInt &operator*=(MInt rhs) & {
61     if (getMod() < (1ULL << 31)) {
62         x = x * rhs.x % int(getMod());
63     } else {
64         x = mul(x, rhs.x, getMod());
65     }
66     return *this;
67 }
68 constexpr MInt &operator+=(MInt rhs) & {
69     x = norm(x + rhs.x);
70     return *this;
71 }
72 constexpr MInt &operator-=(MInt rhs) & {
73     x = norm(x - rhs.x);
74     return *this;
75 }
76 constexpr MInt &operator/=(MInt rhs) & {
77     return *this *= rhs.inv();
78 }
79 friend constexpr MInt operator*(MInt lhs, MInt rhs) {
80     MInt res = lhs;
81     res *= rhs;
82     return res;
83 }
84 friend constexpr MInt operator+(MInt lhs, MInt rhs) {
85     MInt res = lhs;
86     res += rhs;
87     return res;
88 }
89 friend constexpr MInt operator-(MInt lhs, MInt rhs) {
90     MInt res = lhs;
91     res -= rhs;
92     return res;
93 }
94 friend constexpr MInt operator/(MInt lhs, MInt rhs) {
95     MInt res = lhs;
96     res /= rhs;
97     return res;
98 }
99 friend constexpr std::istream &operator>>(std::istream &is, MInt &a) {
100     ll v;
101     is >> v;
102     a = MInt(v);
103     return is;
104 }
105 friend constexpr std::ostream &operator<<(std::ostream &os, const MInt &a) {
106     return os << a.val();
107 }
108 friend constexpr bool operator==(MInt lhs, MInt rhs) {
109     return lhs.val() == rhs.val();
110 }

```

```
111     friend constexpr bool operator!=(MInt lhs, MInt rhs) {
112         return lhs.val() != rhs.val();
113     }
114     friend constexpr bool operator<(MInt lhs, MInt rhs) {
115         return lhs.val() < rhs.val();
116     }
117 };
118
119 template<>
120 ll MInt<0>::Mod = 998244353;
121
122 constexpr ll P = 11(1E18) + 9;
123 using Z = MInt<P>;
124
125 std::mt19937_64 rng(std::chrono::steady_clock::now().time_since_epoch().count());
126 const Z B = rng();
```